

Ściąga SQL++ / N1QL – Couchbase

Kontekst: ściągą jest przygotowana jako pomoc do laboratorium z dokumentowych baz danych na przykładzie Couchbase.

Pracujemy na danych Northwind załadowanych do Couchbase.

Najważniejsze kolekcje:

Kolekcja	Znaczenie
orders	zamówienia
orderdetails	pozycje zamówień
customers	klienci
products	produkty
orders_nested	zamówienia zagnieżdżone, z tablicą items

Pełna ścieżka do kolekcji ma postać:

```
`bucket`.scope.collection
```

W naszym laboratorium:

```
`north0`._default.orders  
`north0`._default.customers
```

Jak korzystać ze ściąg

Ta ściąg pokazuje głównie **wzorce składniowe**, a nie gotowe rozwiązania zadań.

Nazwy takie jak:

```
kolekcja  
alias  
pole  
pole_grupujące  
klucz  
tablica  
element
```

należy zastąpić konkretnymi nazwami z danych Northwind, np.:

```
orders  
orderdetails  
customers  
products  
orders_nested  
OrderID  
CustomerID  
items
```

Przykład:

```
FROM `north0`._default.kolekcja AS alias
```

należy w realnym zapytaniu zamienić np. na:

```
FROM `north0`._default.orders AS o
```

Celem ściąg jest pomoc w składni, a nie zastąpienie samodzielnego rozwiązania zadania.

1. Couchbase jako baza dokumentowa

Couchbase przechowuje dane jako dokumenty JSON.

W klasycznej bazie relacyjnej myślimy zwykle tak:

baza danych → schemat → tabela → wiersz → kolumna

W Couchbase struktura jest inna:

bucket → scope → collection → document → field

Przybliżone porównanie:

Relacyjna baza danych	Couchbase
baza danych	bucket
schemat	scope
tabela	collection
wiersz	document
kolumna	field
klucz główny	document key / META().id

2. Podstawowe SELECT

2.1. Pobranie całego dokumentu

```
SELECT alias
FROM `north0`._default.kolekcja AS alias
WHERE alias.pole IS NOT MISSING
LIMIT 3;
```

2.2. Pobranie wybranych pól

```
SELECT
  alias.pole1,
  alias.pole2
FROM `north0`._default.kolekcja AS alias
WHERE alias.pole1 IS NOT MISSING
LIMIT 10;
```

2.3. Alias kolekcji

Tak jak w SQL, warto używać aliasów:

```
FROM `north0`._default.orders AS o
```

Potem do pól odwołujemy się przez alias:

```
o.OrderID
o.CustomerID
o.OrderDate
```

3. Ścieżka do kolekcji i znaki backtick

W Couchbase nazwa bucketu często zapisywana jest w znakach backtick:

```
`north0`
```

Dlaczego używamy backticków?

- pozwalają bezpiecznie odwołać się do nazw bucketów, scope'ów i kolekcji,
- są szczególnie przydatne, gdy nazwa może zawierać znaki specjalne,
- w materiałach laboratoryjnych stosujemy je konsekwentnie dla czytelności.

4. Liczenie dokumentów

Wzorzec ogólny:

```
SELECT COUNT(1) AS nazwa_licznika  
FROM `north0`._default.kolekcja  
WHERE pole_kluczowe IS NOT MISSING;
```

Przykład zastosowania wzorca:

```
SELECT COUNT(1) AS orders_count  
FROM `north0`._default.orders  
WHERE OrderID IS NOT MISSING;
```

W analogiczny sposób można policzyć dokumenty w innych kolekcjach, zmieniając nazwę kolekcji i pole kontrolne.

5. Identyfikator dokumentu – META().id

Dokument w Couchbase ma własny klucz dokumentu. Nie zawsze jest on tym samym, co pole zapisane wewnątrz dokumentu.

Do pobrania klucza dokumentu używamy:

```
META(alias).id
```

Przykład:

```
SELECT
  META(n).id AS document_id,
  n.OrderID,
  n.CustomerID
FROM `north0`._default.orders_nested AS n
WHERE n.OrderID IS NOT MISSING
LIMIT 5;
```

6. Braki danych: MISSING, NULL, IFMISSINGORNULL

W dokumentowych bazach danych dokumenty nie muszą mieć identycznej struktury. Pole może:

- istnieć i mieć wartość,
- istnieć i mieć wartość NULL,
- nie istnieć w dokumencie w ogóle — wtedy jest MISSING.

6.1. Sprawdzenie, czy pole istnieje

```
WHERE alias.pole IS NOT MISSING
```

Ten warunek jest bardzo ważny w Couchbase, bo często pomaga plannerowi wykorzystać indeks.

6.2. Zastępowanie braku wartości wartością domyślną

Wzorzec:

```
IFMISSINGORNULL(alias.pole, wartość_domyślna)
```

Przykład — brak rabatu traktowany jako 0:

```
IFMISSINGORNULL(od.Discount, 0)
```

6.3. Wzorzec na wartość pozycji zamówienia

```
wartość pozycji = cena * ilość * (1 - rabat)
```

W SQL++:

```
od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))
```

7. Agregacje podstawowe

Cel	Składnia
liczba dokumentów	COUNT(1)
suma	SUM(x)
średnia	AVG(x)
minimum	MIN(x)
maksimum	MAX(x)
liczba unikalnych wartości	COUNT(DISTINCT x)
zaokrąglenie	ROUND(x, 2)

Wzorzec z GROUP BY:

```
SELECT
  alias.pole_grupujące,
  COUNT(1) AS cnt
FROM `north0`._default.kolekcja AS alias
WHERE alias.pole_grupujące IS NOT MISSING
GROUP BY alias.pole_grupujące
ORDER BY cnt DESC;
```

8. Indeksy w Couchbase

To jedna z najważniejszych rzeczy do zapamiętania. W Couchbase zapytanie może się nie wykonać, jeśli nie ma odpowiedniego indeksu. Komunikat błędu może wyglądać mniej więcej tak:

```
No index available on keyspace ...
```

8.1. Primary index

Primary index pozwala uruchomić zapytania eksploracyjne, ale zwykle prowadzi do pełnego skanu kolekcji.

```
CREATE PRIMARY INDEX nazwa_indeksu
ON `north0`._default.kolekcja;
```

Usunięcie:

```
DROP INDEX nazwa_indeksu
ON `north0`._default.kolekcja;
```

Primary index jest wygodny na początku, ale nie jest dobrym rozwiązaniem produkcyjnym.

8.2. Indeks celowy

Indeks celowy tworzymy pod konkretne pole i konkretne zapytanie.

```
CREATE INDEX nazwa_indeksu
ON `north0`._default.kolekcja(pole);
```

W praktyce, aby planner mógł skorzystać z indeksu celowego, często dodajemy warunek na indeksowanym polu:

```
WHERE alias.pole IS NOT MISSING
```

W tym laboratorium ten wzorec będzie pojawiał się często, bo pomaga pisać zapytania zgodne z indeksami.

8.3. Indeksy do łączy

Do JOIN potrzebne są indeksy na polach używanych w warunku łączenia, po stronie dołączanej kolekcji.

Wzorec:

```
CREATE INDEX idx_kolekcja_pole
ON `north0`._default.kolekcja_dołączana(pole_łącznie);
```

W naszym środowisku część indeksów jest tworzona automatycznie podczas inicjalizacji.

9. JOIN w Couchbase

JOIN w Couchbase przypomina składnię SQL, ale działa na dokumentach JSON w kolekcjach.

Nie mamy tu klasycznych relacji wymuszanych przez bazę tak jak w relacyjnych systemach baz danych. Odpowiedzialność za spójność relacji spoczywa bardziej na aplikacji i projekcie danych.

Przed wykonaniem JOIN upewnij się, że istnieje indeks na polu używanym do łączenia po stronie kolekcji dołączanej.

9.1. Wzorzec JOIN dwóch kolekcji

```
SELECT
  a.pole1,
  a.pole2,
  b.pole3
FROM `north0`._default.kolekcjaA AS a
JOIN `north0`._default.kolekcjaB AS b
  ON a.klucz = b.klucz
WHERE a.klucz IS NOT MISSING
      AND b.klucz IS NOT MISSING
LIMIT 10;
```

9.2. Wzorzec JOIN z agregacją

```
SELECT
  a.pole_grupujące,
  SUM(b.wartość) AS suma,
  COUNT(1) AS cnt
FROM `north0`._default.kolekcjaA AS a
JOIN `north0`._default.kolekcjaB AS b
  ON a.klucz = b.klucz
WHERE a.klucz IS NOT MISSING
      AND b.klucz IS NOT MISSING
GROUP BY a.pole_grupujące
ORDER BY suma DESC
LIMIT 10;
```

9.3. Wzorzec JOIN trzech kolekcji

```
SELECT
  a.pole1,
  b.pole2,
  c.pole3
FROM `north0`._default.kolekcjaA AS a
JOIN `north0`._default.kolekcjaB AS b
  ON b.klucz = a.klucz
JOIN `north0`._default.kolekcjaC AS c
  ON c.klucz = a.klucz2
WHERE a.klucz IS NOT MISSING
      AND b.klucz IS NOT MISSING
      AND c.klucz IS NOT MISSING;
```

10. Dokumenty zagnieżdżone

W kolekcji `orders_nested` jeden dokument odpowiada jednemu zamówieniu.

Pozycje zamówienia są przechowywane w tablicy:

```
items
```

Przykładowa uproszczona struktura:

```
{
  "type": "order_nested",
  "OrderID": 10248,
  "CustomerID": "VINET",
  "ShipCountry": "France",
  "items": [
    {
      "ProductID": 42,
      "UnitPrice": 9.8,
      "Quantity": 10,
      "Discount": 0,
      "LineValue": 98
    },
    {
      "ProductID": 11,
      "UnitPrice": 14,
      "Quantity": 12,
      "Discount": 0,
      "LineValue": 168
    }
  ]
}
```

Podjęzienie dokumentu:

```
SELECT alias
FROM `north0`._default.orders_nested AS alias
WHERE alias.OrderID = 10248;
```

Liczba elementów w tablicy:

```
ARRAY_LENGTH(alias.items)
```

11. UNNEST – rozbijanie tablicy w dokumencie

UNNEST służy do rozbicia tablicy znajdującej się w dokumencie.

Jeżeli dokument zamówienia ma tablicę items, to:

```
UNNEST alias.items AS item
```

zamienia jeden dokument zamówienia na wiele rekordów — po jednym dla każdej pozycji zamówienia.

11.1. Wzorzec UNNEST

```
SELECT
  alias.pole1,
  alias.pole2,
  element.pole_a,
  element.pole_b
FROM `north0`._default.kolekcja AS alias
UNNEST alias.tablica AS element
WHERE alias.pole_kluczowe IS NOT MISSING;
```

11.2. Wzorzec UNNEST z agregacją

```
SELECT
  alias.pole_grupujące,
  SUM(element.wartość) AS suma,
  COUNT(1) AS cnt
FROM `north0`._default.kolekcja AS alias
UNNEST alias.tablica AS element
WHERE alias.pole_kluczowe IS NOT MISSING
GROUP BY alias.pole_grupujące
ORDER BY suma DESC
LIMIT 10;
```

12. JOIN vs UNNEST

To najważniejsze porównanie w tym laboratorium.

Sytuacja	Używamy
dane są w osobnych kolekcjach	JOIN
dane są w tablicy wewnątrz dokumentu	UNNEST

To nie jest tylko różnica składniowa. To konsekwencja decyzji projektowej: czy dane trzymamy osobno, czy zagnieżdżamy je w jednym dokumencie.

12.1. Model niezagnieżdżony

Dane są rozbite na osobne kolekcje. Aby je połączyć, trzeba użyć JOIN:

```
FROM kolekcjaA AS a
JOIN kolekcjaB AS b
  ON b.klucz = a.klucz
```

12.2. Model zagnieżdżony

Dane są w jednym dokumencie. Aby rozbić tablicę, trzeba użyć UNNEST:

```
FROM kolekcja AS alias
UNNEST alias.tablica AS element
```

Oba podejścia mogą dać ten sam wynik biznesowy, ale reprezentują inną decyzję projektową.

13. EXPLAIN

EXPLAIN pokazuje plan wykonania zapytania.

```
EXPLAIN
SELECT ...
```

13.1. Najważniejsze elementy planu

Element planu	Znaczenie
IndexScan	baza korzysta z indeksu
PrimaryScan	baza korzysta z primary index, zwykle mniej optymalnie
Fetch	baza dociąga pełny dokument
NestedLoopJoin	wykonanie JOIN
Unnest	rozbicie tablicy z dokumentu
Group	agregacja
Order	sortowanie
Limit	ograniczenie liczby wyników
Project	wybór kolumn/pól zwracanych w wyniku

13.2. Jak czytać plan w uproszczeniu?

Nie trzeba rozumieć każdego szczegółu planu.

Wystarczy odpowiedzieć na pytania:

1. Czy zapytanie korzysta z indeksu?
2. Czy pojawia się PrimaryScan, czy raczej IndexScan?
3. Czy baza musi wykonać JOIN?
4. Czy pojawia się Unnest?
5. Czy na końcu występuje agregacja, sortowanie i limit?

14. Materializacja wyników przez UPSERT INTO ... SELECT

Ta sekcja dotyczy zadania dodatkowego. Nie jest konieczna do wykonania podstawowej części laboratorium. Couchbase pozwala zapisać wynik zapytania do osobnej kolekcji. To może być użyteczne przy budowie warstwy raportowej.

14.1. Utworzenie kolekcji

```
CREATE COLLECTION `north0`._default.nazwa_kolekcji;
```

14.2. Wzorzec zapisu wyników

```
UPSERT INTO `north0`._default.kolekcja_docelowa (KEY k, VALUE v)
WITH dane AS (
  SELECT
    pole_kluczowe AS k,
    pole1,
    pole2
  FROM `north0`._default.kolekcja_zrodlowa
  WHERE pole_kluczowe IS NOT MISSING
  GROUP BY pole_kluczowe, pole1, pole2
)
SELECT
  k,
  {
    "pole1": pole1,
    "pole2": pole2
  } AS v
FROM dane;
```

UPSERT oznacza, że zapytanie można uruchamiać wielokrotnie — istniejące dokumenty zostaną nadpisane.

15. Miniściąga funkcji i konstrukcji Couchbase

Konstrukcja	Co robi	Przykład
<code>META(x).id</code>	zwraca klucz dokumentu	<code>META(n).id</code>
<code>IS NOT MISSING</code>	sprawdza, czy pole istnieje	<code>OrderID IS NOT MISSING</code>
<code>IFMISSINGORNULL(x, y)</code>	zastępuje brak/NULL wartością domyślną	<code>IFMISSINGORNULL(Discount, 0)</code>
<code>ARRAY_LENGTH(x)</code>	liczba elementów tablicy	<code>ARRAY_LENGTH(n.items)</code>
<code>UNNEST</code>	rozbija tablicę na rekordy	<code>UNNEST n.items AS item</code>
<code>COUNT(1)</code>	liczba dokumentów/rekordów	<code>COUNT(1)</code>
<code>COUNT(DISTINCT x)</code>	liczba unikalnych wartości	<code>COUNT(DISTINCT o.OrderID)</code>
<code>ROUND(x, 2)</code>	zaokrąglenie	<code>ROUND(revenue, 2)</code>
<code>CREATE INDEX</code>	tworzy indeks celowy	<code>CREATE INDEX idx ON kolekcja(pole)</code>
<code>CREATE PRIMARY INDEX</code>	tworzy primary index	<code>CREATE PRIMARY INDEX idx ON kolekcja</code>
<code>DROP INDEX</code>	usuwa indeks	<code>DROP INDEX idx ON kolekcja</code>
<code>EXPLAIN</code>	pokazuje plan zapytania	<code>EXPLAIN SELECT ...</code>
<code>UPSERT INTO ... SELECT</code>	zapisuje wynik zapytania do kolekcji	<code>UPSERT INTO kolekcja ... SELECT ...</code>

16. Najważniejsze różnice do zapamiętania

Obszar	Klasyczna baza relacyjna	Couchbase
Jednostka danych	wiersz tabeli	dokument JSON
Struktura danych	zwykle sztywna	elastyczna, dokumenty mogą mieć różne pola
Organizacja	baza/schemat/tabela	bucket/scope/collection
Relacje	zwykle kontrolowane przez bazę	zwykle kontrolowane przez aplikację i projekt danych
Zapytania	SQL	SQL++ / N1QL, SQL-like
Brak pola	zwykle NULL	MISSING albo NULL
Łączenie kolekcji	JOIN	JOIN, ale wymaga indeksów
Dane zagnieżdżone	rzadziej	naturalny element modelu dokumentowego
Tablice w rekordzie/dokumencie	nietypowe	typowe, obsługiwane przez UNNEST
Indeksy	ważne dla wydajności	często konieczne, aby zapytanie w ogóle ruszyło
Plan zapytania	EXPLAIN	EXPLAIN, plan graficzny w UI